18

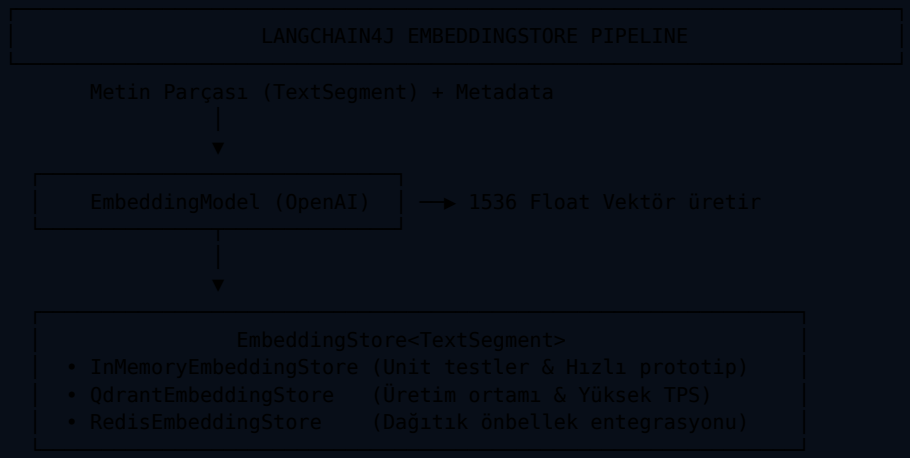
BÖLÜM: VEKTÖRLER, EMBEDDING'LER VE VEKTÖR VERITABANLARI | MODULE: VECTORS, EMBEDDINGS & VECTOR DATABASE ARCHITECTURE

LangChain4j EmbeddingStore: Bellek-İçi ve Dağıtık Vektör Yönetimi

LangChain4j EmbeddingStore: In-Memory, Qdrant & Redis Distributed Vector Management

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Yeni bir yapay zeka özelliği kodlarken her seferinde Docker ile koca bir PostgreSQL veritabanı ayağa kaldırmakla uğraşmak istemezsiniz (Tıpkı birim test yazarken H2 in-memory veritabanı kullanmak gibi). LangChain4j'in `InMemoryEmbeddingStore` yapısı sayesinde tek satır kodla RAM üzerinde çalışan geçici bir vektör veritabanı oluşturabilirsiniz! Testleriniz 1 saniyede biter. Canlıya (Production) çıkarken ise tek bir satırı değiştirip onu devasa bir Qdrant veya pgvector kümesine bağlarsınız!

🇬🇧 ENGLISH (ELI5)

When prototyping a new feature or running unit tests, spinning up external PostgreSQL clusters or Docker containers creates friction. LangChain4j provides `InMemoryEmbeddingStore`: an ephemeral vector store running directly inside JVM heap memory with zero dependencies! Your CI/CD unit tests execute in milliseconds. When shipping to production, you swap a single bean configuration to point to Qdrant or pgvector without touching your application logic!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

LangChain4j Vektör Mimarisi: 1) `TextSegment`: Vektörleştirilen metin parçasıdır; `Metadata` haritası ile zenginleştirilir. 2) `Embedding`: Ham `float[]` dizisini sarmalar. 3) `EmbeddingMatch\<TextSegment>`: Arama sonucunda dönen nesnedir; `score` (benzerlik puanı), `embeddingId` ve `embedded` (orijinal metin) taşır. 4) Serialization: LangChain4j bellek içi vektörleri JSON formatında diske kaydedip tekrar yükleme (`serializeToJsonFile` / `initFromFile`) desteğine sahiptir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

LangChain4j Vector Abstraction Internals: 1) `TextSegment`: Immutable domain object pairing text slices with contextual metadata maps. 2) `Embedding`: Strongly-typed wrapper encapsulating floating-point arrays. 3) `EmbeddingMatch\<TextSegment>`: Result envelope providing similarity scores, vector IDs, and reconstructed segments. 4) Portability & Persistence: The in-memory store can snapshot its graph and vectors directly to local disk via JSON serialization, enabling offline test fixtures.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```java
// Java 25 & LangChain4j: In-Memory Vektör Deposu ve Benzerlik Araması
import dev.langchain4j.data.embedding.Embedding;
import dev.langchain4j.data.segment.TextSegment;
import dev.langchain4j.store.embedding.inmemory.InMemoryEmbeddingStore;
import dev.langchain4j.store.embedding.EmbeddingMatch;
import java.util.List;

void main() {
    var store = new InMemoryEmbeddingStore<TextSegment>();

    // Örnek segment ve vektör ekleme
    var segment1 = TextSegment.from("Kafka, dağıtık bir commit log mimarisine sahiptir.");
    var embedding1 = Embedding.from(new float[]{0.9f, 0.1f, 0.0f});
    store.add(embedding1, segment1);

    var segment2 = TextSegment.from("PostgreSQL, ilişkisel ve ACID uyumlu bir veritabanıdır.");
    var embedding2 = Embedding.from(new float[]{0.0f, 0.9f, 0.1f});
    store.add(embedding2, segment2);

    // Benzerlik araması: Kafka'ya yakın sorgu
    var queryEmbedding = Embedding.from(new float[]{0.85f, 0.15f, 0.0f});
    List<EmbeddingMatch<TextSegment>> matches = store.findRelevant(queryEmbedding, 1, 0.70);

    for (var match : matches) {
        IO.println("En Yakın Eşleşme (Skor: " + match.score() + "): " + match.embedded().text());
    }
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Hafıza Uyarısı: `InMemoryEmbeddingStore` milyonlarca vektör için uygun DEĞİLDİR! Her 1536 boyutlu float vektör bellekte yaklaşık 6KB yer kaplar. 1 milyon vektör 6 GB saf RAM tüketir ve JVM Garbage Collector'ı kitleyebilir (GC Pauses).

⚠️ Hallucination & Interview Trap

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

Memory threshold: Do not run `InMemoryEmbeddingStore` in production with >100,000 vectors. Storing dense 1,536-dimensional float arrays in heap memory causes massive JVM footprint and triggers fatal garbage collection pauses.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• InMemoryEmbeddingStore, birim testler ve hızlı prototipleme için benzersizdir.

• LangChain4j'in tip güvenli yapısı (TextSegment, EmbeddingMatch) temiz kod sağlar.

• Büyük ölçekli üretim ortamlarında mutlaka disk ve bellek optimizasyonlu harici motorlar kullanılmalıdır.

🎯 Key Takeaways for the Interview

• InMemoryEmbeddingStore is unbeatable for rapid prototyping and hermetic unit testing.

• Type-safe wrappers (TextSegment, EmbeddingMatch) ensure clean architectural decoupling.

• Production systems mandate external, specialized vector engines with persistence.